Homework-3

> **!** Important
>
> This homework is due Sunday, Nov 1st at 11:59pm.
> Late window Monday, Nov 2nd at 11:59pm.

You will turn in a PDF file (generated by Quarto) to Gradescope. It is important that you select pages correctly when turning in your assignment. Please watch the video: *For Students: Submit Homework on the Gradescope Website* on our Moodle page. Part of your grade is turning in your assignment correctly.

Formatting

– Your pdf should be generated by Quarto. Do not Render to html and print to PDF.

– Each question should be labeled with a level-2 section header

– If the question has multiple parts, you should create a level-4 section header with the appropriate label (ex. a)

– Each question should be on a new page. Type `\newpage` to create a new page after each question

– **Your code should not run off the page**. Check your document for any formatting issues before submitting.

Version Control

> **Note**
>
> Part of your grade comes from demonstrating proper use of version control.
> You must have 4 meaningful commits to earn full credit.
> Each commit should be after you answer a specific task.

Set up

Clone your hw-3 GitHub repo on GitHub.

If you do not have the necessary packages installed for this assignment, please install them.

Packages

```
library(tidyverse)
library(gapminder)
library(gganimate)
```

Question 1 (50-points)

Please watch the following video on data visualization here .

After you have watched the entire video, please answer the following questions.

The data used in the video are pre-loaded into R, and called `gapminder`. You are going to both re-create a still image, and the animated version of his plot. Here are some things to consider:

The x-axis uses a logarithmic scale in the video. At the beginning, around 0:30, he introduces income values of $400, $4,000, and $40,000. Those values are equally spaced on the graph even though each is ten times the previous value. We can transform our x-axis to be on a logarithmic scale by using the function `scale_x_log10()` with no other arguments.

To hide the legends, as seen in the video, please see the help file for `theme()` and search the `legend.position =` argument.

Note: The `gapmider` data does not have any information on the Middle East. You may exclude this information for the purpose of this project.

a

Pause the video when Hans Rosling is showing data for the year 1977. Screenshot this image, and embed this into your quarto document. Note: put this image in an images folder in your repo, write the correct file path to embed the picture.

Underneath the picture, define the following:

– x-axis = Income

– y-axis = Lifespan

– size of the dots = population size

In addition, define the number of regions Hans is showing, and share the color that each represent.

Europe - Brown

Asia - Red

Middle East - Green (gapminder doesn't have these data)

Africa (south of Sahara) - Blue

Americas - Yellow

b

In the `gapminder` data set, there are 5 total regions in the `continent` column. Our goal is to match the Hans plot as best as possible. To do this, we need to combine `Oceania` and `Asia` together.

– Create a new variable called region that changes "Oceania" to "Asia", using if_else(). Save and overwrite the existing gapminder data set.

– Create and show a contingency table of the variable region to show that there are only 4 unique regions in your data set.

```
gapminder <- gapminder |>
  mutate(
    region = if_else(continent == "Oceania", "Asia", continent)
  )

gapminder |>
  count(region)
```

```
# A tibble: 4 x 2
  region       n
  <chr>    <int>
1 Africa     624
2 Americas   300
3 Asia       420
4 Europe     360
```

c

In the same code chunk for question c…

– create a new data set called gapminder1977 that only has data for the year 1977.

– use these data to recreate the plot you pasted in part a.

– You do not need to have the large Year value in your plot. Instead, put this in your title

– The axes do not need to have the same font, and can be the generic axes labels outside of the plot

– You do not need "years" under the y-axis labels.

– The colors must match. You may reference the generic color names when setting the colors.

– The y-axis must show the values 25, 50, and 70. We can do this by setting arguments in scale_y_continuous(). Please see the arguments breaks = and limits =.

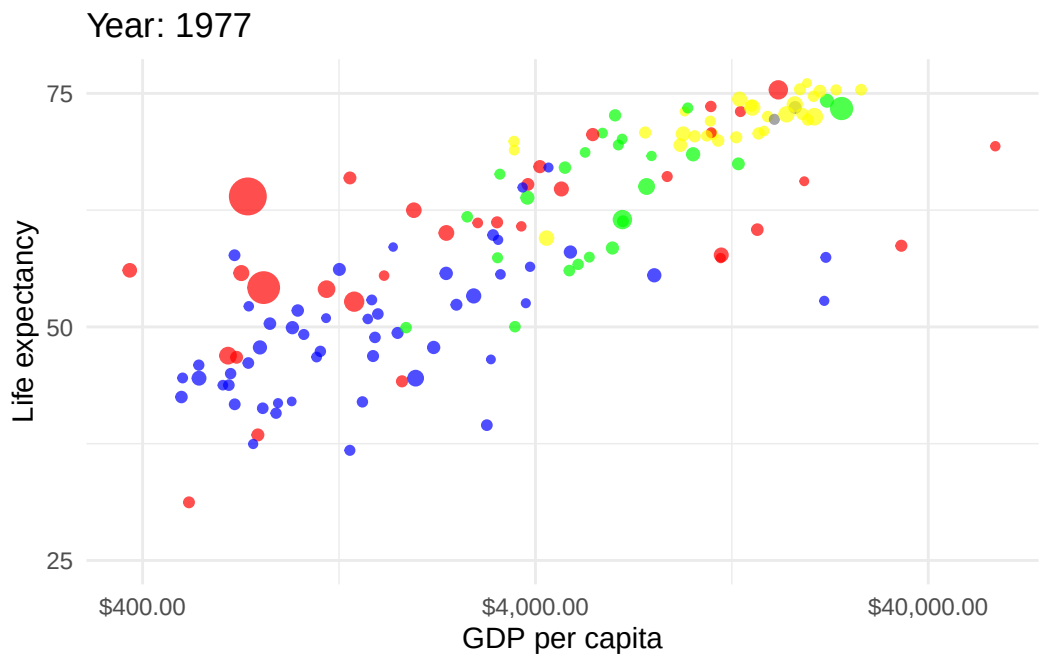
– Your x-axis should show 400, 4,000, and 40,000. You can set this in the scale_x_log10() function using the breaks = argument.

– Your x-axis should have $ before 400, 4,000, and 40,000. You can set this using the labels argument in the scale_x-log10() function. Note: label_dollar() is in the scales package. Directly call the label_dollar() function from the scales package. Do not library scales for this assignment.

```
gapminder1977 <- gapminder |>
  filter(year == 1977)

ggplot(
  gapminder1977,
  aes(
    x = gdpPercap,
    y = lifeExp,
    size = pop,
    color = continent
  )
) +
  geom_point(alpha = 0.7) +
  scale_x_log10(
  breaks = c(400, 4000, 40000),
  labels = scales::label_dollar()
) +
  scale_y_continuous(
  breaks = c(25, 50, 75),
  limits = c(25, NA)
) +
  labs(
    title = "Year: 1977",
    x = "GDP per capita",
    y = "Life expectancy",
    size = "Population",
    color = "Continent"
```

```
  ) +
  theme_minimal() +
  theme(legend.position = "none") +
 scale_color_manual(
  values = c(
    "Americas" = "green",
    "Europe" = "yellow",
    "Africa" = "blue",
    "Asia" = "red"
  )
)
```



d

You will give many presentations as a statistician/data scientist/researcher. Communicating data to a general audience is one of the most important skills one can develop. Reflecting on the video, please answer the following questions.

– What are two/three ways that Hans Rosling was effective in communicating the story of the data?

– What are two/three strategies you may try and implement during a future presented?

Question 2 (10-points)

Now, we are going to re-create his animation! Please note that when you render to a PDF, your plot will not animate. Animation only works on non-static documents like a HTML document.

To do this, we are going to…

– Copy your code from question-1 part c into a new code chunk here. Save your plot as a R object, p.

– use the function `transition_time()` with the argument of your time variable you want your plot to animate through. This needs to be layered onto your `ggplot()` canvas.

– Make sure you are using the entire data set, and not the filtered data set

– Update the title of your plot to be "Year: {frame_time}". This creates a dynamic title where frame_time is updated according to the transition_time() variable.

– This will not be supported as a pdf. Make sure to set your code chunk to echo: true and eval: false so we can still see your code. You should be able to see / enjoy your animation in the Viewer pane!

– You will see some warnings due to values being removed because they started to fall outside the limits set. This is okay for this assignment!

Note: You need to use the function `animate()` to actually see your animated image. For example: `animate(p, fps = 10, duration = 10)`. Take a look at the help file for `animate()` for more information.

Mac users

Mac users need an additional two packages for their plot to run. Please run the following code:

```
install.packages("gifski")
```

```
install.packages("transformr")
```

Make sure you also library each package.

You need to run the following code to see your plot!

```
animate(p, renderer = gifski_renderer())
```

```r
p <- ggplot(
  gapminder,
  aes(
    x = gdpPercap,
    y = lifeExp,
    size = pop,
    color = region
  )
) +
  geom_point(alpha = 0.7) +
  scale_x_log10(
  breaks = c(400, 4000, 40000),
  labels = scales::label_dollar()
) +
  scale_y_continuous(
  breaks = c(25, 50, 75),
  limits = c(25, NA)
) +
  labs(
    title = "Year: {frame_time}",
    x = "GDP per capita",
    y = "Life expectancy",
    size = "Population",
    color = "Region"
  ) +
  theme_minimal() +
  theme(legend.position = "none") +
 scale_color_manual(
  values = c(
    "Americas" = "green",
    "Europe" = "yellow",
    "Africa" = "blue",
    "Asia" = "red"
  )
) +
  transition_time(year)

animate(p, fps = 10, duration = 10)
```